One Slow Client Must Never Freeze Safety

Project Hands-on Day 3: Serve Without Blocking

`transport_v1` $\longrightarrow$ `runtime_poll_v1`

Mentor – Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors – Mr. Jiang Suyu, Mr. Ruan Haozhe

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

`transport_v1` must expose all four timestamps

Run once before editing:

```
make verify-day2
```

If the gate is red, record the failure and switch to the latest class-generated `transport_v1` checkpoint. Do not repair MQTT during this hour.

The runtime work may begin only if:

- normal, duplicate, and delayed fixtures run;
- `t_pub`, `t_rx`, `t_gate`, and `t_ack` appear in raw output;
- invalid or stale input still produces zero actuator writes.

```
Frozen:  FrameV1 / topics / QoS / timebase / freshness verdicts
```

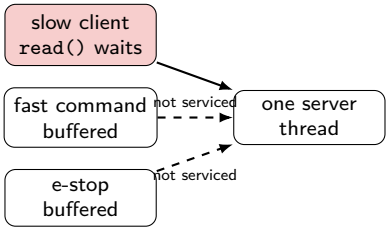
A blocking read lets one client hold the robot

Reproduce head-of-line blocking:

```
make demo-day3-blocking
```

Soda Code

```
loop {
    let (mut c, _) = listener.accept()?;
    loop {
        let n = c.read(&mut buffer)?;
        if n == 0 { break; }
        handle(&buffer[..n]); // then
    waits on c
    }
}
```

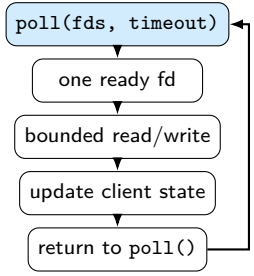


Record the first interval in which fast/e-stop bytes are ready but the server is sleeping on the slow fd.

Failure: fast/safety tail tracks the slow pause.

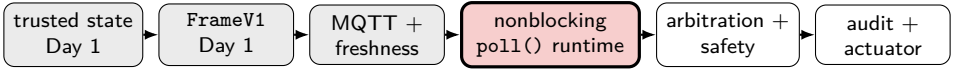
Nonblocking I/O, `poll()`, and backpressure bound the damage

1. Mark accepted fds `O_NONBLOCK`; empty I/O returns `EAGAIN` instead of sleeping.
2. Use `poll()` to service only ready fds.
3. Keep per-client buffers for partial frames and partial writes.
4. Cap work, queued bytes, and queue age per loop turn; apply the frozen overload policy.



Invariant: no callback blocks; each ready fd receives at most `WORK_BUDGET` per turn.

Today replaces one blocking edge—not the transport contract



Writable today

`runtime_poll`, `client_state`, buffer limits, slow/flood fixtures, runtime logs.

Read-only today

State and frame schemas, topics, QoS, timestamp meanings, freshness verdicts, simulator API.

Input `transport_v1` ⟶ **Output** `runtime_poll_v1`

Each team observes blocking at a different boundary

Team	Build	Deliver	Pass
G1	Feed one state stream to fast and slow consumers	`freshness-clients.csv`	Slow does not delay fast state
G2	Connect bridge to fast/slow/flood clients; retain four stamps	`queue-location.csv`	Delay source is locatable
G3	Add nonblocking + `poll()`; run 1/10/50 clients	`runtime_poll_v1` + before/after data	Slow does not affect fast or safety path
G4	Measure slow-client/HOL delay on the safety path	`priority-risk.csv`	Risk is quantified with tail data
G5	Replace the integrated blocking runtime with G3's `poll()` runtime	`integrated-poll.log`	Slow blocks neither action nor safety

Run the same workload before and after `poll()`

Soda Code

```
loop {
    // Project wrapper backed by poll().
    for event in poller.wait(TICK_MS)? {
        if !event.is_readable() { continue; }
        let id = event.client_id();
        let client = &mut clients[id];
        let bytes = client.read_ready(INBUF_SIZE)?;
        client.input.extend_from_slice(&bytes);
        // TODO 1: retain partial FrameV1 in client.
        // TODO 2: dispatch at most WORK_BUDGET.
        // TODO 3: cap bytes and queue age.
    }
}
```

Compare exactly:

- blocking baseline;
- `poll()` method;
- 1, 10, and 50 clients;
- fast, slow, and flood fixtures.

At minute 11: green, blocked, or using class checkpoint.

The fix passes when slow no longer controls fast

Run the canonical gate:

```
make verify-day3
```

Save `blocking-vs-poll.csv`, `queue-policy.json`, and `manifest.json` inside `Gxx_D3_evidence/`.

The gate is green only when all three claims are true:

1. fast and safety paths remain within their frozen tail-latency budgets while the slow client pauses;
2. the 1/10/50-client before/after table includes p50, p95, p99, maximum, queue age, and workload label;
3. invalid/stale rejection and the four transport timestamps remain unchanged after the runtime replacement.

Freeze workload, seed, repetitions, metric definitions, and the eight-minute benchmark command; add 2–3 method bullets and one main-result caption per chapter.

Tomorrow, many producers must share one actuator safely

Input `transport_v1` ⟶ **Output** `runtime_poll_v1`

Freeze and hand off:

- per-client state and work budget;
- maximum queue bytes and age;
- fast/slow/flood before/after data.

Before tagging:

- name the artifact owner;
- record one known issue or `none`;
- verify G4/G5 run the workload.

Tag only after green: `git tag -a runtime_poll_v1 -m "Day 3 gate passed"`

Day 4 preflight: `make verify-day3`

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template